NYC Restaurant Location Recommender

Ranks available storefronts in New York City for a restaurant concept. Two models feed the ranking:

1. **Survival model (XGBoost):** how likely is any restaurant at this spot to still be open after 3 years? It looks only at the location, never at cuisine.
2. **Cuisine gap score:** is the area around this spot busy for the chosen cuisine (demand) while the spot itself is not (supply)?

Rent and space requirements are **filters**: listings that don't fit are removed before scoring and never affect the scores.

Contents

1. Quick start
2. Folder layout
3. Pipeline overview
4. Step by step: what each script does
5. The survival model in detail
6. The cuisine gap score in detail
7. How a listing is scored
8. Recommender options
9. Output files
10. Settings you can tune
11. Known limitations
12. Troubleshooting
13. Data sources

---

1. Quick start

Install the packages once (in the Miniforge terminal):

```
python -m pip install pandas numpy h3 xgboost scikit-learn scipy pyarrow
```

Put the input files in `E:\440 CAPSTONE\data` (see Folder layout), then:

```
cd "E:\440 CAPSTONE\codes"
python run_all.py --cuisine "Indian" --max-monthly-rent 9000
```

That builds everything (about 30 seconds) and writes a ranked list to `outputs\recommendations\recommendations_indian.csv`.

For each new concept after that, run only the recommender; nothing needs rebuilding:

```
python recommend.py --cuisine "Coffee / Tea" --boroughs Brooklyn --top 15
```

---

2. Folder layout

```
E:\440 CAPSTONE\
├── codes\       the scripts (this README lives here)
├── data\        raw inputs you download
└── outputs\     everything the scripts write (created automatically)
```

All paths are set in `paths.py`. To move the project, change the one `ROOT` line there, or set the `CAPSTONE_ROOT` environment variable.

Scripts look for each input in `outputs\` first and then in `data\`, so files made by an earlier step are found automatically.

Inputs (`data\`)

File	What it is	Where it comes from
`current_restaurants.csv`	Every currently permitted NYC food establishment, latest inspection per restaurant	DOHMH Restaurant Inspection Results (NYC Open Data `43nn-pn8j`), cleaned to one row per `camis`
`restaurants_raw.parquet`	67,790 restaurants from 2015–2026, including closed ones, with first/last snapshot dates	nyc-cuisine-atlas (`data/restaurants_raw.parquet`). `run_all.py` downloads it automatically if missing
`master_tract_features.csv`	One row per 2020 census tract (2,327) with demographic, job, transit, built-environment and market features	Built by the team; see the Master Tract Feature Table Spec
`synthetic_properties.csv`	Available storefronts to rank (synthetic scenarios, not real listings)	Built by the team

Code (`codes\`)

Script	Role
`paths.py`	Folder locations, shared by every script
`label_cuisines.py`	Step 1: assigns each restaurant one of 50 cuisines
`cuisine_matrix.py`	Step 2: 50 × 50 table of how much cuisines compete with each other
`make_heatmaps.py`	Step 3: hex heatmaps and the cuisine gap score
`build_lifespans.py`	Step 4: when each restaurant opened and closed
`features.py`	Shared feature preparation (used by steps 5 and 6)
`train_survival.py`	Step 5: trains the XGBoost survival model
`recommend.py`	Step 6: filters and ranks listings for a concept
`run_all.py`	Runs the steps in order

3. Pipeline overview

Steps 1–5 build the models and only need re-running when input data changes. Step 6 runs once per restaurant concept.

---

4. Step by step: what each script does

Step 1: `label_cuisines.py`

DOHMH uses about 90 cuisine codes, several of them vague ("Latin American", "Caribbean", "Chicken", blank). This script maps every restaurant to one of **50 project cuisines**.

How a label is chosen, in order:

1. **Direct:** the DOHMH code maps cleanly (e.g. "Thai" → Thai, "Hamburgers" → Burgers).
2. **Specialty override:** a generic code is refined by the name. An "American" restaurant called "Golden Diner" becomes Diner; an "Italian" pizzeria becomes Pizza.
3. **Keyword:** vague or blank codes are matched on the restaurant name ("El Cibao" → Dominican, "Pho Bang" → Vietnamese).
4. **Default:** a vague code with no keyword gets a best guess ("Latin American" → Dominican, "Caribbean" → Jamaican).

5. **Unclassified:** nothing to go on (mostly blank codes for never-inspected caterers and hotels).

The `label_source` column records which rule was used (`direct`, `keyword`, `default`, `weak_keyword`, `none`), so guessed labels can be excluded from analysis.

Cuisines outside the 50 go to their nearest neighbour: Ecuadorian and Bolivian → Peruvian; Salvadoran, Guatemalan and Honduran → Mexican; Nepali and Tibetan → Indian.

Step 2: `cuisine_matrix.py`

A symmetric 50 × 50 matrix `M`, values 0–1, saying how much a restaurant of cuisine B competes with one of cuisine A.

- Diagonal = 1 (same cuisine).
- About 92% of pairs are 0 (independent).
- Examples: Indian ↔ Pakistani/Bangladeshi 0.8, Dominican ↔ Puerto Rican 0.7, Chinese ↔ Taiwanese 0.6, Italian ↔ Pizza 0.5, Japanese ↔ Korean 0.25.

All values are listed by hand in the `PAIRS` list at the top of the script, grouped by region, so each one can be defended or changed.

Step 3: `make_heatmaps.py`

Works on an **H3 resolution-9 hex grid**: each hex is about 0.105 km², roughly 400 m across, or 4–6 city blocks. That matches a 5–10 minute walk, the distance most diners travel for a casual meal.

For each hex and cuisine it computes:

Column	Meaning
`count`	Restaurants of this cuisine in the hex
`density_smoothed`	Count spread to neighbouring hexes (own hex 1.0, ring 1 0.5, ring 2 0.15)
`competition_weighted`	Smoothed counts of all cuisines, weighted by the matrix
`local_competition`	Matrix-weighted restaurants per hex right here (see section 6)
`area_competition`	Matrix-weighted restaurants per hex in the surrounding 0.8–1.6 km
`gap_score`	0–1, high where the area is busy for this cuisine but this spot is not
`hex_has_restaurants`	False for hexes with no restaurant at all (often water or parks)

It also writes `cuisine_heatmaps.html`, an interactive map with three views: **Same cuisine**, **Competition** and **Gaps**. Open it in Chrome or Edge. The basemaps (Esri streets, light gray, satellite, OpenStreetMap) need no API key.

Step 4: `build_lifespans.py`

The live DOHMH dataset only lists restaurants that are open today, so it can't show closures. The nyc-cuisine-atlas project saved 13 copies of the dataset from the Internet Archive (Sept 2015 – Apr 2026) plus the current data. A restaurant missing from the newest copy has closed.

Column	Meaning
`open_date`	First pre-permit inspection (the true opening), else first inspection seen
`open_known`	True if `open_date` came from a pre-permit inspection
`closed`	True if missing from the latest snapshot
`close_date`	Midpoint between the last snapshot listing it and the next one
`close_window_days`	Width of that gap, i.e. how uncertain `close_date` is
`years`	Open date → close date (or → latest snapshot if still open)
`clean_cohort`	True if opened after 2020-11-06 with a known opening date

Only `clean_cohort` rows are used for training. No snapshots exist between April 2017 and November 2020, so restaurants that opened and closed in that gap are missing entirely, which would make earlier cohorts look healthier than they were.

Step 5: `train_survival.py`

Trains the XGBoost survival model. See section 5.

Step 6: `recommend.py`

Filters the listings, scores the survivors and ranks them. See sections 7 and 8.

`run_all.py`

Runs steps 1–5 in order, plus step 6 when `--cuisine` is given. Before each step it checks that the step's input files exist, and it stops at the first failure with the command to resume.

```
python run_all.py                       build everything
python run_all.py --cuisine "Thai"      build everything, then recommend
python run_all.py --from train          resume from a step
python run_all.py --only heatmaps       run one step
python run_all.py --list                list the steps
```

Options that `run_all.py` doesn't recognise (`--boroughs`, `--chain`, `--top`, …) are passed through to `recommend.py`.

---

5. The survival model in detail

What it predicts

The probability that a restaurant opened at a given location is **still open 3 years later**. It ignores cuisine on purpose: the survival model judges the location, and the gap score judges the cuisine fit.

One row per restaurant, not per area

Training rows are individual restaurants (about 18,900 opened after November 2020), not tracts or hexes. Most areas only see a few openings, so an area-level survival rate would swing on one closure. Area quality enters through the features instead.

Target: survival time as an interval

A restaurant's closure is only known to fall **between two snapshots**, and many restaurants are still open (censored). XGBoost's `survival:aft` (accelerated failure time) objective accepts exactly this:

Restaurant	Lower bound	Upper bound
Closed	Years from opening to the last snapshot listing it	Years to the next snapshot
Still open	Years from opening to the latest snapshot	Infinity

A plain "closed within 3 years: yes/no" target would have to throw away every restaurant opened in the last 3 years; AFT keeps them.

The model predicts a median survival time `T`. With a normal error distribution of scale σ = 1 on the log scale, the 3-year survival probability is:

```
P(open at 3 years) = 1 − Φ( (ln 3 − ln T) / σ )
```

Features

Built in `features.py` from the master tract table, plus three per restaurant.

Tract features (47 columns; 50 with the restaurant features below):

Group	Features
Residents	Population density, % under 18, % 65+, median household income (blanks filled from neighbouring tracts), income-was-missing flag
Workers	Job density, jobs as a share of residents + jobs, % jobs in retail, food & accommodation, office industries, high-earning
Walkability	EPA national walkability index, smart location score
Subway	Ridership per sq mi, station count, distance to nearest station, its ridership
Citi Bike	Departures per sq mi (0 where no station), evening and weekend share, distance to nearest station
Uber/Lyft	Drop-offs and pickups per sq mi of taxi zone
Pedestrians	Count at nearest DOT counter, distance to it
Built environment	Retail and office floor area per sq mi, % commercial or mixed lots, vacant land share, average floors, storefronts per sq mi, storefront vacancy rate
Food market	Food & nightlife POIs, bars, cafés per sq mi (OpenStreetMap)
Events	Permitted events in the next 90 days
Neighbours	Average over bordering tracts of 8 key features (`*_nbr`), plus neighbour population density, jobs per resident and income from the master

Restaurant features:

Feature	Meaning
`is_chain`	DBA name appears at 5+ locations
`competitors_400m_at_open`	Restaurants of any cuisine open within 400 m on the opening date
`open_year`	Year opened (captures post-COVID cohort effects)

Deliberately excluded:

- **Cuisine:** handled by the gap score.
- **Rent:** a filter, not a predictor.
- **DOHMH current-restaurant columns** (active counts, grades, inspection scores): they only describe restaurants that survived, so they would leak the answer.

Fixes applied to the master table

`features.py` corrects known issues in the master at load time:

Problem	Fix
Every tract carries its whole taxi zone's drop-off total	Divide by the zone's total land area to get a density
Citi Bike columns blank where a tract has no station	Set to 0
Storefront vacancy column empty	1 − leased ÷ registered storefronts
`jobs_per_resident` explodes in tracts with almost no residents	Use jobs ÷ (jobs + residents) instead
Restaurant locations have no tract ID	Assign the tract with the nearest centroid

Fairness across boroughs

Manhattan has far more restaurants than the Bronx or Staten Island, so an unweighted model would mostly learn Manhattan. Each training row is weighted so **every tract-income quintile contributes equally**.

Validation

5-fold cross-validation **grouped by neighbourhood (NTA)**: each fold is tested on neighbourhoods the model never saw. Random splits would overstate accuracy, because neighbouring tracts are near-copies of each other.

Results with the current data:

Borough	Restaurants	C-index
Overall	18,860	0.563
Bronx	1,552	0.542
Brooklyn	5,146	0.543
Manhattan	7,108	0.573
Queens	4,363	0.558
Staten Island	691	0.553

The **C-index** is the share of restaurant pairs the model ranks in the right order of closing: 0.5 is random, 1.0 perfect. About 0.56 is typical for a location-only model, because whether one particular restaurant fails depends mostly on how it is run. Read the scores as comparisons between locations, not as a forecast for one restaurant.

The strongest predictors are opening year, chain vs independent, neighbouring and local income, Citi Bike activity, subway ridership and late-night Uber/Lyft drop-offs. The full list is in `outputs\model\feature_importance.csv`.

Scoring new locations

New listings are scored as if opened in **2023**, the latest year with a full 3 years of follow-up. Later cohorts are mostly still open, so the model knows little about them yet.

---

6. The cuisine gap score in detail

Assumption: demand for a cuisine is where that cuisine (or related ones) already competes. The best spots are quiet pockets inside or between busy areas, not empty areas.

For each hex `h` and cuisine `c`:

1. Count and weight by the matrix. `C[h, c]` = restaurants of cuisine `c` in hex `h`.

```
W[h, c] = Σ over c' of  C[h, c'] × M[c', c]
```

For Pakistani, an Indian restaurant counts 0.8 and a Thai one counts 0.

2. Local supply ("here"): own hex plus ring 1 (the 6 touching hexes, ~400 m), ring 1 weighted 0.25, expressed per hex:

```
local = ( W[own] + 0.25 × Σ W[ring 1] ) / 2.5
```

3. Area demand ("nearby"): the 54 hexes in rings 2–4 (~0.8–1.6 km), weighted equally:

```
area = Σ W[rings 2, 3, 4] / 54
```

4. Combine:

```
demand     = area / (95th percentile of area for this cuisine), capped at 1
saturation = local / area          (set to 1 when area = 0)
gap        = demand × (1 − saturation), clipped to 0–1
```

Demand is scaled per cuisine, so small cuisines aren't measured against American's numbers.

Situation	local	area	gap
Quiet band between dense clusters	low	high	**high**
Middle of a dense cluster	high	high	~0
Empty outskirts	~0	~0	~0
Average neighbourhood	≈ area	moderate	low

Water and parks: a river hex beside dense blocks looks like a perfect gap. The map only draws gaps in hexes that already contain a restaurant, and the recommender only scores real storefronts. If you rank gaps from the CSV yourself, filter `hex_has_restaurants == True` first.

---

7. How a listing is scored

`recommend.py` takes a restaurant concept and runs four stages:

1. Filter. Remove listings that fail any hard requirement: transaction type, rent, price, size, borough, ground floor, existing food use.

2. Survival. For each remaining listing:

- tract features from the listing's `geoid`,
- `is_chain` from the concept,
- `competitors_400m_at_open` = restaurants open within 400 m today,
- `open_year` = 2023.

The model gives `survival_3yr`. `survival_vs_borough` is that value minus the median of the borough's other listings, so a strong Bronx spot stands out even though Manhattan averages higher.

3. Cuisine gap. The listing's coordinates → res-9 hex → `gap_score` for the chosen cuisine from `hex_cuisine_scores.csv`.

4. Combine.

```
score = w_survival × (percentile rank of survival_3yr among filtered listings)
      + w_cuisine  × cuisine_gap
      + bonuses
```

Defaults: `w_survival = 0.6`, `w_cuisine = 0.4`. Optional bonuses: +0.05 for a former food space, +0.03 for possible outdoor seating, +0.03 for a corner unit.

Survival uses its percentile rank, not the raw probability, because predicted probabilities fall in a narrow band: about 48–80%, with half of all listings between 65% and 70%. On the raw scale, survival would barely move the ranking next to the 0–1 gap score.

---

8. Recommender options

```
python recommend.py --cuisine "Indian" [options]
python recommend.py --list-cuisines
```

Option	Effect
Concept	
`--cuisine NAME`	One of the 50 cuisines (required). Partial names work: `"coffee"` → Coffee / Tea
`--chain`	Concept is a chain or franchise
Filters	
`--transaction lease|purchase|any`	Default `lease`
`--max-monthly-rent N`	Maximum monthly rent (leases)
`--max-rent-psf N`	Maximum annual rent per sq ft (leases)
`--max-price N`	Maximum purchase price (purchases)
`--min-sqft N`, `--max-sqft N`	Space size
`--boroughs A B ...`	e.g. `--boroughs Bronx Queens`
`--ground-floor-only`	Exclude second-floor units
`--require-food-use`	Only spaces previously used for food
Preferences	
`--prefer-food-use`	+0.05 for former food spaces
`--prefer-outdoor`	+0.03 for possible sidewalk seating
`--prefer-corner`	+0.03 for corner units
Weights and output	
`--w-survival X`, `--w-cuisine Y`	Score weights (defaults 0.6 / 0.4)
`--top N`	Rows printed (default 20; the CSV always has every listing)

Examples:

```
python recommend.py --cuisine "Thai" --max-monthly-rent 8000 --min-sqft 1000
python recommend.py --cuisine "Coffee / Tea" --chain --boroughs Bronx --top 10
python recommend.py --cuisine "Pizza" --transaction purchase --max-price 1500000
python recommend.py --cuisine "Dominican" --require-food-use --w-cuisine 0.6 --w-
survival 0.4
```

---

9. Output files

```
outputs\
├── restaurants_labeled.csv        step 1: restaurants with cuisine_50 and label_source
├── cuisine_matrix.csv             step 2: 50 × 50 matrix
├── hex_cuisine_scores.csv         step 3: one row per (hex, cuisine), ~28 MB
├── cuisine_heatmaps.html          step 3: interactive map
├── restaurant_lifespans.csv       step 4: open/close dates per restaurant
├── model\
│   ├── survival_model.json        trained XGBoost model
│   ├── model_meta.json            feature list, AFT scale, reference year
│   ├── cv_report.txt              cross-validation results by borough
│   ├── feature_importance.csv     gain importance per feature
│   └── tract_survival_scores.csv  3-year survival per tract (independent restaurant)
└── recommendations\
    └── recommendations_<cuisine>.csv
```

Columns in `recommendations_<cuisine>.csv`:

Column	Meaning
`property_id`, `borough`, `neighborhood`, `transaction`	From the listing
`available_sqft`, `monthly_rent`, `annual_rent_psf`, `asking_price`	From the listing
`former_food_use`, `corner_position`	From the listing
`survival_3yr`	Model's 3-year survival probability
`survival_vs_borough`	Difference from the borough's median listing
`restaurants_within_400m`	All restaurants open within 400 m today
`cuisine_here`	Local matrix-weighted competition for the cuisine
`cuisine_nearby_area`	Area competition (demand proxy)
`cuisine_gap`	Gap score, 0–1
`score`	Final ranking score
`latitude`, `longitude`, `geoid`	Location

`tract_survival_scores.csv` is ready for a map layer: each tract's 3-year survival for an independent restaurant with today's competition, plus its percentile within its borough.

10. Settings you can tune

Setting	File	Default	Effect
`HORIZON_YEARS`	`train_survival.py`	3.0	Survival horizon
`PARAMS`	`train_survival.py`	depth 5, lr 0.03, min_child_weight 25	XGBoost hyperparameters
`COMPETITION_RADIUS_M`	`features.py`	400	Radius for counting competitors
`NEIGHBOUR_COLS`	`features.py`	8 columns	Features that get a neighbour average
`chain_names(min_locations=5)`	`features.py`	5	Locations needed to count as a chain
`RING_WEIGHTS`	`make_heatmaps.py`	1.0 / 0.5 / 0.15	Heatmap display smoothing
`LOCAL_RING1_WEIGHT`	`make_heatmaps.py`	0.25	How much ring 1 counts as "here"
`AREA_RINGS`	`make_heatmaps.py`	(2, 3, 4)	How far out demand is measured
`DEMAND_PERCENTILE`	`make_heatmaps.py`	95	Area level that counts as full demand
`PAIRS`	`cuisine_matrix.py`	~100 pairs	Cuisine substitution values
Keyword rules	`label_cuisines.py`		How names map to cuisines
`--w-survival`, `--w-cuisine`	`recommend.py` (command line)	0.6 / 0.4	Score weights

After changing anything in steps 1–5, rerun from that step, e.g. `python run_all.py --from heatmaps`.

---

11. Known limitations

- **Survival is location-only and modestly predictive** (C-index about 0.56). Operator quality, menu and price matter more than location for any single restaurant.
- **The gap score measures supply, and infers demand from it.** A cuisine that has never been tried in an area scores zero there even if residents would want it. A demand model based on ACS ancestry data would address this.
- **Closure dates are uncertain** by the gap between snapshots: 3–10 months since 2023, about 3.5 years for 2017–2020 (which is why training starts in November 2020).
- **Very short-lived restaurants can be missed** if they open and close between two snapshots, so first-year survival is overstated.
- **A change of owner looks like a closure** because the new owner gets a new CAMIS ID.
- **Most features are 2023–2026 snapshots,** while training restaurants opened 2020–2026, so features describe the area slightly after some restaurants opened.
- **Events use upcoming permits,** not past ones; past-year counts would be better.
- **Restaurants are placed in the tract with the nearest centroid,** not by exact boundaries, which can misassign points near tract edges.
- **Listings are synthetic scenarios,** not real availability.

---

12. Troubleshooting

Problem	Fix
`ModuleNotFoundError: No module named 'h3'` (or `xgboost`, `pyarrow`)	`python -m pip install h3 xgboost pyarrow` in the same terminal you run the scripts from
`ModuleNotFoundError: No module named 'paths'` (or `features`, `label_cuisines`)	All `.py` files must be together in `codes\`
VS Code Run button does nothing	Select the Miniforge interpreter (`Ctrl+Shift+P` → Python: Select Interpreter → `C:\Users\athar\miniforge3\python.exe`), or run from the terminal
`Step 'x' can't run: missing ...`	Put the listed file in `data\`, or run the earlier steps
Parquet download fails	Download it from the link printed in the error, save it in `data\`, then `python run_all.py --from lifespans`
`Unknown cuisine`	`python recommend.py --list-cuisines`
Cuisine gap is 0 everywhere	`hex_cuisine_scores.csv` is missing; run `python run_all.py --only heatmaps`
Map shows hexes but no streets	Open the HTML directly in a browser; in-app previews block map tiles
`0 of 1000 listings pass the filters`	Loosen the filters; note `--transaction` defaults to `lease`

---

13. Data sources

- DOHMH New York City Restaurant Inspection Results, NYC Open Data
- nyc-cuisine-atlas: Wayback Machine snapshots of the DOHMH dataset, 2015–2026
- Master tract table sources (ACS, LODES, MTA, Citi Bike, TLC, DOT, PLUTO, EPA Smart Location, DOF storefronts, OpenStreetMap, permitted events): see the Master Tract Feature Table Spec

- H3 hexagonal grid, Uber
- XGBoost survival analysis (AFT)